CONFIDENTIAL · SECURITY AUDIT

VERSION 4 · RE-REVIEW

Security Audit Tech for Palestine Website

techforpalestine.org · Cloudflare Pages & Workers

This is the **fourth-round re-review**, assessing patches applied after v3 (2026-04-06). Six more findings have been resolved, bringing the total fixed count to **16 of the original 22 issues**. No critical or high-severity findings remain from the original list. This report documents one new medium-severity finding in the newly added server-side proxy and the few remaining open items.

V1 ORIGINAL	V2 RE-REVIEW	V3 RE-REVIEW	V4 RE-REVIEW
2026-03-31	2026-04-02	2026-04-06	2026-04-09

Executive Summary — v4 Re-review

Delta audit of patches applied after v3 (2026-04-06). All critical and high findings are now resolved.

16	0	0	2	3	5
TOTAL FIXED	CRITICAL	HIGH	MEDIUM	LOW	OPEN

Remediation progress: 16 of 22 findings resolved (73%)

73%

All **0 critical** and **0 high** severity issues remain — a significant milestone. The main outstanding item is the absence of rate limiting (open since v1), which is a configuration-level fix in Cloudflare's dashboard requiring no code changes. A new medium-severity finding was introduced in the server-side proxy: it accepts arbitrary path parameters, allowing any caller to route authenticated requests to any endpoint on the upstream API.

COMPLETE ISSUE REGISTER — V4 STATUS

ID	Finding	Peak Severity	v4 Status
C-1	API key & account ID in wrangler configs	CRITICAL	Fixed v2
C-2	Unauthenticated membership invite endpoint	CRITICAL	Fixed v2
C-3	Unverified webhook triggers privileged invite	CRITICAL	Fixed v2
N-1	Webhook secret exposed in URL query param	CRITICAL	Fixed v3
N-2	Timing attack on token comparison	HIGH	Fixed v3
H-1	Secret key logged to browser console	HIGH	Fixed v3
H-2	Full donor PII payload echoed in HTTP response	HIGH	Fixed v4 ✓
H-3	Middleware caches POST/API responses as public	HIGH	Fixed v3
H-5	Wildcard CORS on all endpoints	HIGH	Fixed v3
N-3	membership-invite proxies arbitrary body	MEDIUM	Fixed v3
M-1	No CSRF / Origin check on form endpoints	MEDIUM	Fixed v4 ✓

ID	Finding	Peak Severity	v4 Status
M-2	No input format/length validation on forms	MEDIUM	Fixed v4 ✓
M-3	getEnv globalThis fallback	MEDIUM	Fixed v3
M-4	Donor PII (email) in server logs	MEDIUM	Fixed v4 ✓
M-5	Internal error.message in HTTP responses	MEDIUM	Fixed v3
R-1	PUBLIC_SECRET_KEY in client-side Axios bundle	MEDIUM	Fixed v4 ✓
R-2	CSP 'unsafe-inline' defeats XSS protection	MEDIUM	Fixed v4 ✓
L-1	Notion DB IDs in wrangler.toml	LOW	Fixed v3
P-1	project-proxy forwards auth to any upstream path (NEW)	NEW MEDIUM	New
H-4	No rate limiting on any endpoint	HIGH	Open
L-2	QGiv embed script without SRI hash	LOW	Open
L-3	Image proxy has no authentication	LOW	Open
L-4	style-src still uses 'unsafe-inline' in CSP	LOW	Open

CONFIRMED FIXED SINCE V3

- **H-2 — Payload PII echo removed** (`qgiv-webhook.ts:184–190`): The fallthrough response now returns only `{"success": true, "message": "Webhook received but donation type unclear"}` . The raw `payload` object is no longer included.
- **M-1 — Origin header check added to both form endpoints**: `endorsement-request.ts:8–13` and `e4p-pledge-sign.ts:8–13` now verify `Origin === "https://techforpalestine.org"` and return HTTP 403 before processing any body content.
- **M-2 — Input validation added to both form endpoints**: Email validated with regex; organization website, campaign link, and LinkedIn URL validated with `new URL()` ; all free-text fields capped at 2,000 characters. Applied to both `endorsement-request.ts` and `e4p-pledge-sign.ts` .
- **M-4 — Email addresses redacted in webhook logs**: All `console.log` / `console.warn` / `console.error` calls in `qgiv-webhook.ts` now log `[redacted]@domain.com` (domain portion only) instead of the full email address.
- **R-1 — Client-side secret removed from browser bundle**: `src/store/api.ts` now points to `/api/project-proxy` with no `Authorization` header. A new server-side route (`src/pages/api/project-proxy.ts`) adds the credential server-side. The secret is no longer accessible to any browser visitor.
- **R-2 — CSP nonce implementation replaces 'unsafe-inline'**: A new `src/middleware.ts` generates a per-request `crypto.randomUUID()` nonce, uses Cloudflare's `HTMLRewriter` to inject it into every `<script>` tag, and sets `script-src 'nonce-{N}' 'strict-dynamic'` . The `Content-Security-Policy` is set dynamically per response, not statically in `_headers` .

NEW FINDING INTRODUCED BY V4 PATCH

NEW MEDIUM P-1

project-proxy Forwards Authorization to Any Arbitrary Upstream Path

File: src/pages/api/project-proxy.ts:27–36 **OWASP:** A01:2021 – Broken Access Control

The new server-side proxy correctly moves the `SECRET_KEY` out of the browser bundle. However, the `path` query parameter is validated only for the presence of a leading `/` before being concatenated to `apiUrl` and forwarded with full authorization. This means any browser visitor can use the proxy as an authenticated gateway to *any endpoint* on the upstream API — not only the intended form endpoints.

```
// project-proxy.ts:27-36
const path = url.searchParams.get("path");

if (!path || !path.startsWith("/")) {  // only checks leading slash
  return new Response(...400...);
}

// Any authenticated path on the upstream API is reachable:
const upstream = `${apiUrl.replace(/\/$/, "")}${path}`;  // ← no allowlist

const headers = new Headers(request.headers);
headers.set("Authorization", secretKey);  // ← added to ALL paths
```

ATTACK SCENARIO

A user on the `/project-apply` page opens DevTools and crafts a request to access an admin endpoint:

```
fetch("/api/project-proxy?path=/api/method/admin.users.getAll")
  // → forwarded as: GET {API_URL}/api/method/admin.users.getAll
  //   with Authorization: {SECRET_KEY}  ← full access
```

Additionally, the proxy forwards the complete set of incoming request headers from the browser (minus `host`) to the upstream API. If the upstream API uses cookies or other client-supplied headers for secondary authorization decisions, those are also forwarded.

SECURE FIX — ALLOWLIST PERMITTED PATHS

```
// project-proxy.ts — replace the path validation block
const ALLOWED_PATHS = new Set([
  "/api/method/your_app.form.get_fields",
  "/api/method/your_app.form.submit",
  // add every legitimate endpoint here
]);

const path = url.searchParams.get("path");

// Normalise: strip query string before allowlist check
const pathOnly = path ? path.split("?")[0] : "";
```

```
if (!path || !ALLOWED_PATHS.has(pathOnly)) {
  return new Response(JSON.stringify({ error: "Path not permitted" }), {
    status: 403,
    headers: { "Content-Type": "application/json" },
  });
}
```

Also remove all unnecessary headers from the upstream request (replace `new Headers(request.headers)` with a minimal set: `Content-Type` and the added `Authorization` only). Do not forward browser-originated headers such as `Cookie`, `X-Forwarded-For`, or custom auth headers.

LOW L-4 style-src Still Uses 'unsafe-inline' in CSP

File: src/middleware.ts:24 **OWASP:** A05:2021 – Security Misconfiguration

`script-src` is now correctly protected by per-request nonces. However, `style-src 'unsafe-inline'` remains. CSS injection is lower risk than script injection, but in some browser versions combined with clever attribute selectors, injected styles can be used to exfiltrate data (e.g., via `background-image: url()` on attribute matches). The primary fix is to move inline styles to external stylesheets or use hashes.

```
// middleware.ts:24 — residual unsafe-inline
"style-src 'self' 'unsafe-inline' https://fonts.googleapis.com",
```

REMAINING OPEN FINDINGS

HIGH H-4 No Rate Limiting on Any Endpoint — Open Since v1

File: All src/pages/api/* endpoints **OWASP:** A04:2021 – Insecure Design

Note: Configuration-level fix only — no code changes needed

Every write endpoint — creating Notion records, triggering Hub invitations, subscribing email addresses — accepts unlimited requests per IP. An attacker can flood `/api/endorsement-request` or `/api/e4p-pledge-sign` at full throughput to fill Notion databases with spam entries or exhaust API quotas. This has been the last remaining high-severity finding since v3 and requires only a Cloudflare dashboard configuration.

FIX — CLOUDFLARE WAF RATE LIMITING (FREE TIER, ~10 MIN TO CONFIGURE)

```
Cloudflare Dashboard → Security → WAF → Rate Limiting Rules

Rule 1 — Write endpoints (Notion forms):
  Expression : (http.request.method eq "POST")
               and starts_with(http.request.uri.path, "/api/")
  Action     : Block
  Threshold  : 10 requests / 60 seconds / IP

Rule 2 — Webhook (should only receive QGiv server traffic):
  Expression : starts_with(http.request.uri.path, "/api/qgiv-webhook")
  Action     : Block
  Threshold  : 30 requests / 60 seconds / IP
```

For the webhook, additionally consider configuring Cloudflare's IP allowlist to accept requests only from QGiv's published server IP ranges.

LOW L-2 QGiv Payment Embed Script Loaded Without Subresource Integrity

File: src/components/MembershipPage.tsx **OWASP:** A08:2021 – Software & Data Integrity Failures

The QGiv embed script (`https://secure.qgiv.com/resources/core/js/embed.js`) is dynamically injected into the DOM without an `integrity` attribute. A supply-chain compromise of QGiv's CDN could deliver malicious JavaScript into the payment context. Compute a SHA-384 hash of the known-good script version and add it as the `integrity` attribute.

```
// Compute: openssl dgst -sha384 -binary embed.js | openssl base64 -A
script.integrity = "sha384-<computed-hash>";
script.crossOrigin = "anonymous";
```

LOW L-3 Notion Image Proxy Has No Authentication

File: cloudflare-worker/worker.js **OWASP:** A01:2021 – Broken Access Control

The worker validates decoded URLs against known Notion S3 domain patterns, preventing open-relay abuse. Any actor with a valid Notion S3 URL can use the proxy to cache and serve that image indefinitely. Low severity since the images are already public via Notion's own CDN. No immediate action required — a shared token header check would provide defence-in-depth if ever desired.

SECURITY POSTURE PROGRESS — ALL ROUNDS

Round	Date	Critical	High	Medium	Low	Total Open	Fixed
v1	2026-03-31	3	5	6	3	17	—
v2	2026-04-02	1	6	7	3	17	3
v3	2026-04-06	0	2	5	3	10	10
v4	**2026-04-09**	**0**	**1**	**2**	**3**	**6**	**16**

PRIORITISED REMEDIATION PLAN — V4

Priority	Action	ID	Effort
Immediate	Configure Cloudflare WAF rate limiting on POST and webhook endpoints	H-4	10 min (dashboard only)
Day 1	Add path allowlist to `project-proxy.ts`; strip all non-essential forwarded headers	P-1	1–2 hours
Week 1	Replace `style-src 'unsafe-inline'` with nonce or hash (move inline styles to external sheets)	L-4	2–4 hours
Week 1	Add SRI `integrity` hash to QGiv embed script injection in `MembershipPage.tsx`	L-2	30 min
Backlog	Consider adding a shared token to image proxy for defence-in-depth	L-3	1 hour

GOOD PRACTICES CONFIRMED IN V4 CODEBASE

- **CSP nonce implementation is correct and complete:** Per-request nonce via `crypto.randomUUID()`; Cloudflare `HTMLRewriter` injects nonce into all `<script>` tags; `strict-dynamic` propagates trust to dynamically loaded scripts; set on HTML responses only (API routes are unaffected).
- **Server-side proxy cleanly removes secret from browser bundle:** Client `api.ts` has no credentials; proxy adds `Authorization` server-side; `transfer-encoding` and `connection` hop-by-hop headers stripped from upstream responses.
- **Form endpoints now have layered validation:** Origin check (CSRF) → presence check → email format → URL format → length limits → Notion write. Correct order of operations.

- **Webhook PII handling is clean:** No full emails in logs (domain-only), no payload echo in responses, no error details leaked to callers.
- **constantTimeEqual utility is idiomatic and correct:** `TextEncoder` + `timingSafeEqual` from `node:crypto`; used consistently in all authentication checks.
- **Middleware handles cache correctly:** Non-GET requests and all `/api/*` routes get `no-store`; CSP is applied to HTML responses only — avoids polluting JSON API responses.
- **All secrets sourced from Cloudflare runtime env bindings** in production; no secrets committed to version control.
- **Zero critical or high severity issues remain** across the entire codebase for the first time since the audit began.